LOGICAL DESIGN HOMEWORKS

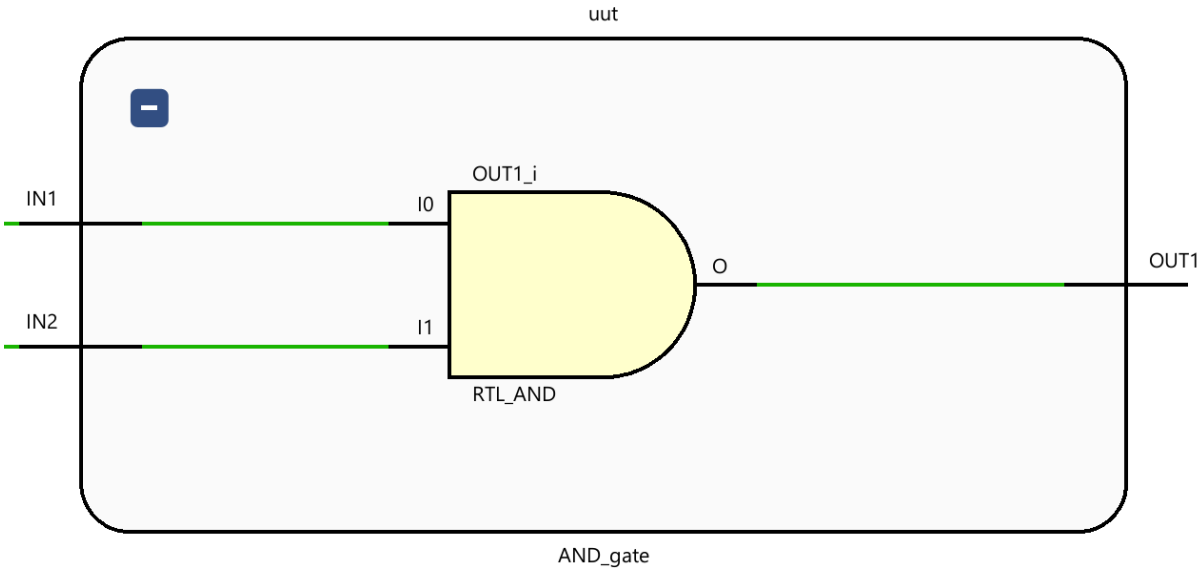
Homework I

Required Task:

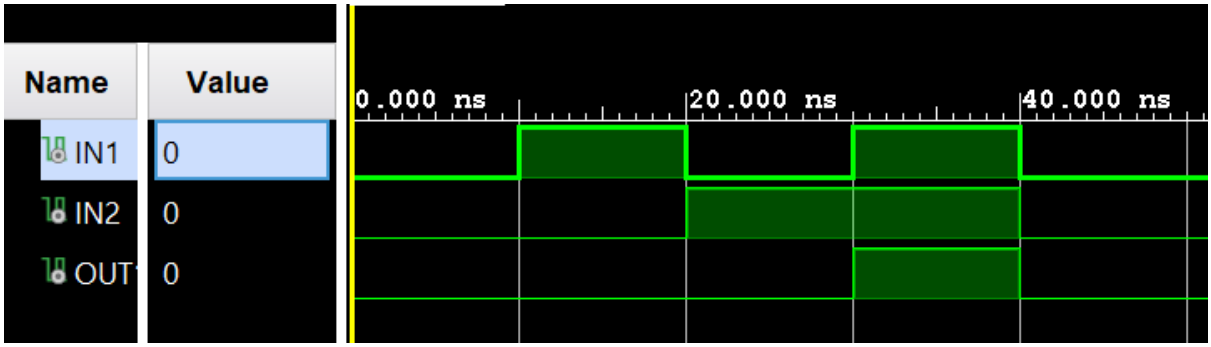
Basics, to setup Vivado, create a project and add tb and definition files.

Running the simulation and RTL Schematic.

RTL Schematic of the AND Gate



Simulation



As it can be seen we have the single AND gate simulaton. Each iput is given 10ns seperrately and then they both have HIGH. Only HIGH output can be seen when both inputs are HIGH.

D

Design Module

```
module AND_gate(a,b,c);
input a,b; //a,b defined as inputs
output c; // c is defined as output.

assign c = a & b; // c will get an output of `logical and` between a-b.

endmodule

```

Testbench

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

ENTITY AND_gate_tb IS --creates an component to represent and gate
END AND_gate_tb;

ARCHITECTURE behavior OF AND_gate_tb IS
    -- the work flow of entire testbenc is written
    -- Component Declaration for the Unit Under Test (UUT)
    COMPONENT AND_gate --I/O of the AND gate are declared here
    PORT(
         IN1 : IN  std_logic;
         IN2 : IN  std_logic;
         OUT1 : OUT  std_logic
        );
    END COMPONENT;
   -- external signals are defined
   --Inputs
   signal IN1 : std_logic := '0';
   signal IN2 : std_logic := '0';

    --Outputs
   signal OUT1 : std_logic;
BEGIN
    --external signals and port of the gate are matched
    -- Instantiate the Unit Under Test (UUT)
   uut: AND_gate PORT MAP (
          IN1 => IN1,
          IN2 => IN2,
          OUT1 => OUT1
        );

    -- insert stimulus here
    -- all comboiations are tested here
    -- every wait holds the current value for 10 seconds
     process
     begin
     wait for 10 ns;

     IN1 <= '1';
     IN2 <= '0';

     wait for 10 ns;

     IN1 <= '0';
     IN2 <= '1';

    wait for 10 ns;

     IN1 <= '1';
     IN2 <= '1';

     wait for 10 ns;

     IN1 <= '0';
     IN2 <= '0';

      wait;
   end process;

END;
```

Homework II

a_1	a_0	b_1	b_0	c_2	c_1	c_0
0	0	0	0	0	0	0
0	0	0	1	0	0	1
0	0	1	0	0	1	0
0	0	1	1	0	1	1
0	1	0	0	0	0	1
0	1	0	1	0	1	0
0	1	1	0	0	1	1
0	1	1	1	1	0	0
1	0	0	0	0	1	0
1	0	0	1	0	1	1
1	0	1	0	1	0	0
1	0	1	1	1	0	1
1	1	0	0	0	1	1
1	1	0	1	1	0	0
1	1	1	0	1	0	1
1	1	1	1	1	1	0

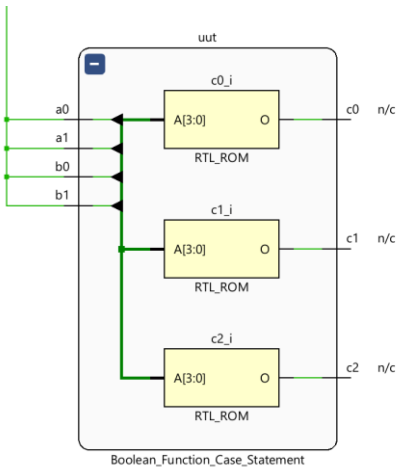
Required Task:

Implementing the given truth table using:
1) case method
2) data flow method

RTL Schematic of the Logical Circuit

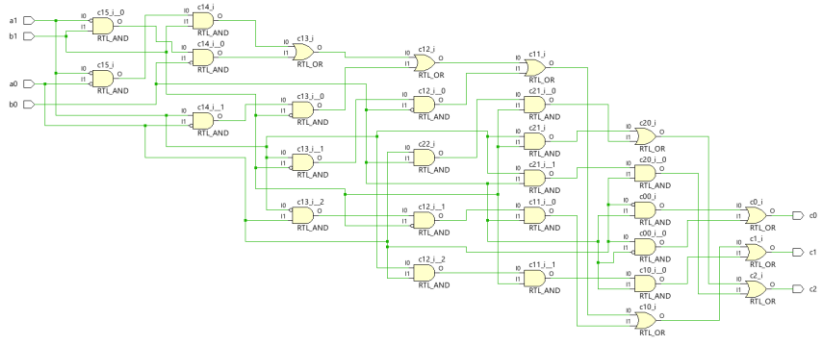
1)case

We have complex logic blocks for each output.

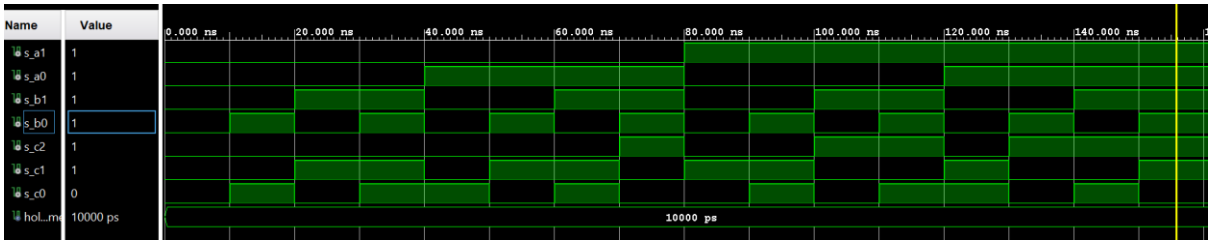


2)dataflow

It implemented each gate individually; I don't know why it did not use a 6 port or gate?

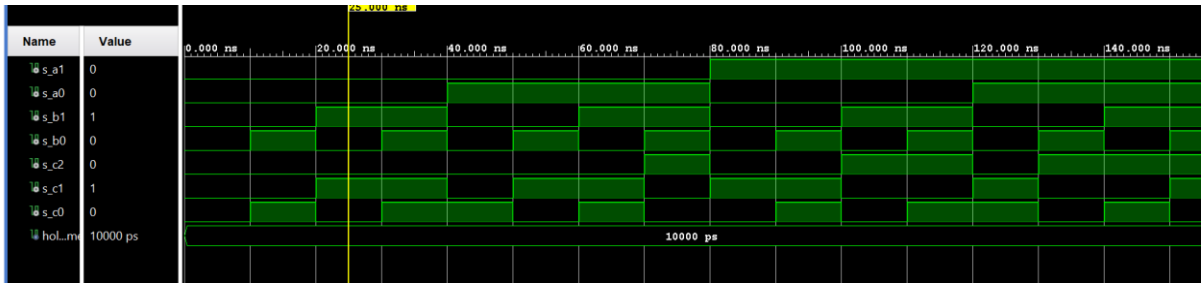


Simulation



We have the following inputs and outputs as parallel to the truth table.

We have the same simulation diagrams for both method, as expected.



Design Module

1)case

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Boolean_Function_Case_Statement is
    Port (
    a0  : in STD_LOGIC;
    a1  : in STD_LOGIC;
    b0  : in STD_LOGIC;
    b1  : in STD_LOGIC;

    c0 : out STD_LOGIC;
    c1 : out STD_LOGIC;
    c2 : out STD_LOGIC);

end Boolean_Function_Case_Statement;

architecture Behavioral of Boolean_Function_Case_Statement is
signal inputs : std_logic_vector(3 downto 0 );
begin
    inputs <= a1&a0&b1&b0;
    process(inputs)
    begin
        case inputs is
                    when "0000" =>
                        c2 <= '0'; c1 <= '0'; c0 <= '0';
                    when "0001" =>
                        c2 <= '0'; c1 <= '0'; c0 <= '1';
```

```
                    when "0010" =>
                        c2 <= '0'; c1 <= '1'; c0 <= '0';
                    when "0011" =>
                        c2 <= '0'; c1 <= '1'; c0 <= '1';
                    when "0100" =>
                        c2 <= '0'; c1 <= '0'; c0 <= '1';
                    when "0101" =>
                        c2 <= '0'; c1 <= '1'; c0 <= '0';
                    when "0110" =>
                        c2 <= '0'; c1 <= '1'; c0 <= '1';
                    when "0111" =>
                        c2 <= '1'; c1 <= '0'; c0 <= '0';
                    when "1000" =>
                        c2 <= '0'; c1 <= '1'; c0 <= '0';
                    when "1001" =>
                        c2 <= '0'; c1 <= '1'; c0 <= '1';
                    when "1010" =>
                        c2 <= '1'; c1 <= '0'; c0 <= '0';
                    when "1011" =>
                        c2 <= '1'; c1 <= '0'; c0 <= '1';
                    when "1100" =>
                        c2 <= '0'; c1 <= '1'; c0 <= '1';
                    when "1101" =>
                        c2 <= '1'; c1 <= '0'; c0 <= '0';
                    when "1110" =>
                        c2 <= '1'; c1 <= '0'; c0 <= '1';
                    when "1111" =>
                        c2 <= '1'; c1 <= '1'; c0 <= '0';
                    when others =>
                        -- Default case, assign all outputs to '0'
                        c2 <= '0'; c1 <= '0'; c0 <= '0';
                end case;
            end process;

end Behavioral;
```

2)dataflow

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Boolean_Function_Data_Flow is
    Port (
    a0  : in STD_LOGIC;
    a1  : in STD_LOGIC;
    b0  : in STD_LOGIC;
    b1  : in STD_LOGIC;

    c0 : out STD_LOGIC;
    c1 : out STD_LOGIC;
    c2 : out STD_LOGIC);

end Boolean_Function_Data_Flow;

architecture dataflow of Boolean_Function_Data_Flow is
begin
        c0 <= (((not a0) and b0) or (a0 and(not b0)));

        c1 <= (((not a1)and(not a0)and b1) or
               ((not a1)and b1 and(not b0)) or
               (a1 and (not a0)and(not b1)) or
               (a1 and(not b1)and(not b0)) or
               ((not a1)and a0 and(not b1)and b0) or
               (a1 and a0 and b1 and b0));

       c2 <=((a1 and b1) or (a0 and b0 and b1) or (a1 and b0 and a0));

      end dataflow;
```

Testbench

1)case

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.numeric_std .ALL;

ENTITY Boolean_Function_Case_Statement_tb IS --creates an component to represent and gate
END Boolean_Function_Case_Statement_tb;

ARCHITECTURE behavior OF Boolean_Function_Case_Statement_tb IS

    COMPONENT Boolean_Function_Case_Statement --I/O of the AND gate are declared here
        Port (
        a0  : in STD_LOGIC;
        a1  : in STD_LOGIC;
        b0  : in STD_LOGIC;
        b1  : in STD_LOGIC=

        c0 : out STD_LOGIC;
        c1 : out STD_LOGIC;
        c2 : out STD_LOGIC);

        end COMPONENT;
```

```
   -- external signals are defined
   --Inputs
-- Inputs (signals to drive your design)
    signal s_a1 : STD_LOGIC;
    signal s_a0 : STD_LOGIC;
    signal s_b1 : STD_LOGIC;
    signal s_b0 : STD_LOGIC;

    -- Outputs (signals to read from your design)
    signal s_c2 : STD_LOGIC;
    signal s_c1 : STD_LOGIC;
    signal s_c0 : STD_LOGIC;
    constant hold_time : time := 10 ns;

BEGIN
    --external signals and port of the gate are matched
          -- Instantiate the Unit Under Test (UUT)
   uut: Boolean_Function_Case_Statement
        PORT MAP (
       a1 => s_a1,
       a0 => s_a0,
       b1 => s_b1,
       b0 => s_b0,
       c2 => s_c2,
       c1 => s_c1,
       c0 => s_c0
);
     -- insert stimulus here
     -- all comboiations are tested here
     -- every wait holds the current value for 10 seconds
     stim_proc: process
        variable v_input_combination : std_logic_vector (3 downto 0);
     begin
           for i in 0 to 15 loop
           v_input_combination := std_logic_vector(to_unsigned(i,4));
           s_a1 <= v_input_combination(3);
           s_a0 <= v_input_combination(2);
           s_b1 <= v_input_combination(1);
           s_b0 <= v_input_combination(0);

           wait for hold_time;
        end loop;
      wait;
   end process;

END;
```

2)dataflow

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.numeric_std .ALL;

ENTITY Boolean_Function_Data_Flow_tb IS --creates an component to represent and gate
END Boolean_Function_Data_Flow_tb;

ARCHITECTURE behavior OF Boolean_Function_Data_Flow_tb IS
    -- the work flow of entire testbenc is written
    -- Component Declaration for the Unit Under Test (UUT)
    COMPONENT Boolean_Function_Data_Flow --I/O of the AND gate are declared here
        Port (
        a0  : in STD_LOGIC;
        a1  : in STD_LOGIC;
        b0  : in STD_LOGIC;
        b1  : in STD_LOGIC;

        c0 : out STD_LOGIC;
        c1 : out STD_LOGIC;
        c2 : out STD_LOGIC);

        end COMPONENT;
   -- external signals are defined
   --Inputs
-- Inputs (signals to drive your design)
    signal s_a1 : STD_LOGIC;
    signal s_a0 : STD_LOGIC;
    signal s_b1 : STD_LOGIC;
    signal s_b0 : STD_LOGIC;

    -- Outputs (signals to read from your design)
    signal s_c2 : STD_LOGIC;
    signal s_c1 : STD_LOGIC;
    signal s_c0 : STD_LOGIC;
    constant hold_time : time := 10 ns;

BEGIN
    --external signals and port of the gate are matched
          -- Instantiate the Unit Under Test (UUT)
   uut: Boolean_Function_Data_Flow
        PORT MAP (
       a1 => s_a1,
       a0 => s_a0,
       b1 => s_b1,
       b0 => s_b0,
       c2 => s_c2,
```

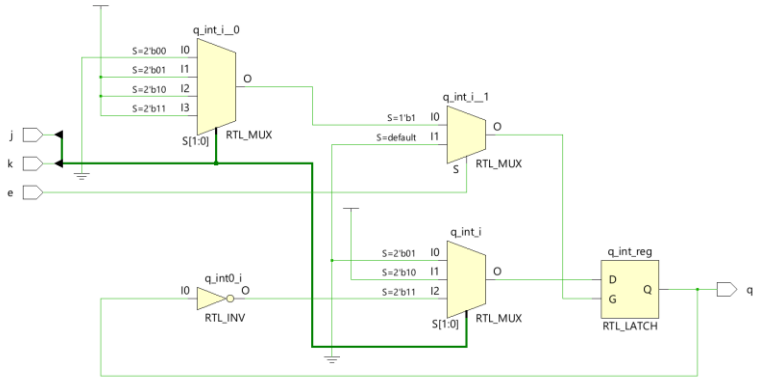
```
       c1 => s_c1,
       c0 => s_c0
);
     -- insert stimulus here
     -- all comboiations are tested here
     -- every wait holds the current value for 10 seconds
     stim_proc: process
        variable v_input_combination : std_logic_vector (3 downto 0);
     begin
            for i in 0 to 15 loop
            v_input_combination := std_logic_vector(to_unsigned(i,4));
            s_a1 <= v_input_combination(3);
            s_a0 <= v_input_combination(2);
            s_b1 <= v_input_combination(1);
            s_b0 <= v_input_combination(0);

            wait for hold_time;
        end loop;
      wait;
   end process;
END;
```

Homework V

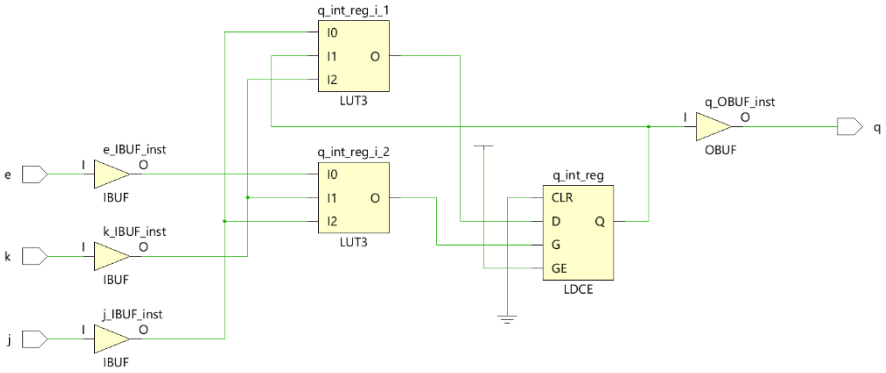
Part I)

We are asked to design a JK-Latch. I used the case method to design the latch.

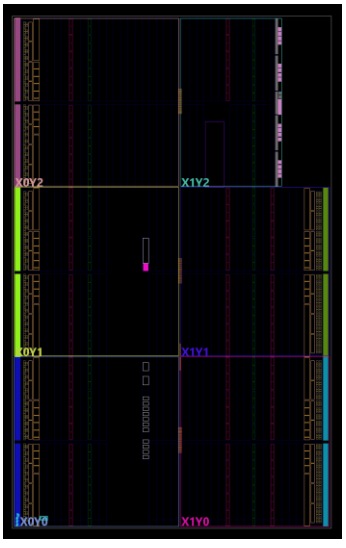
RTL Schematic



Implementation Schematic



Implementation on FPGA / xc7a35tftg256-1



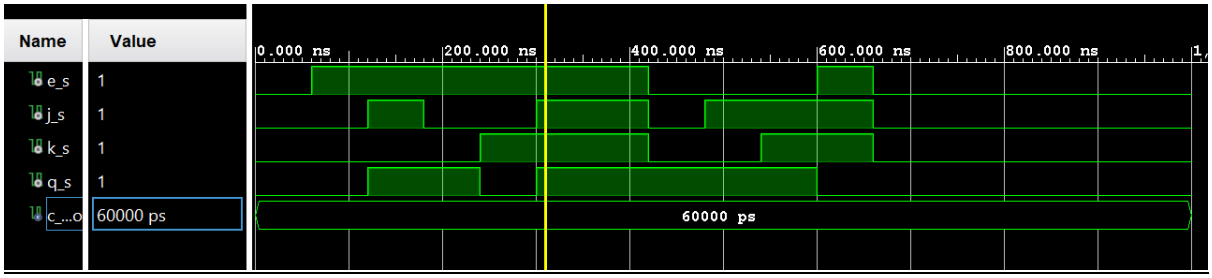
Simulation

Behavioral

Here it can be seen that there is a problem with double toggling at 360ns. This is caused because, the system only processes the inputs when there is a change in the given parameters inside process (). Since values e, j, and k are stable, the system does not apply any changes to the Q. The solve this, we can add a momentarily change such as:

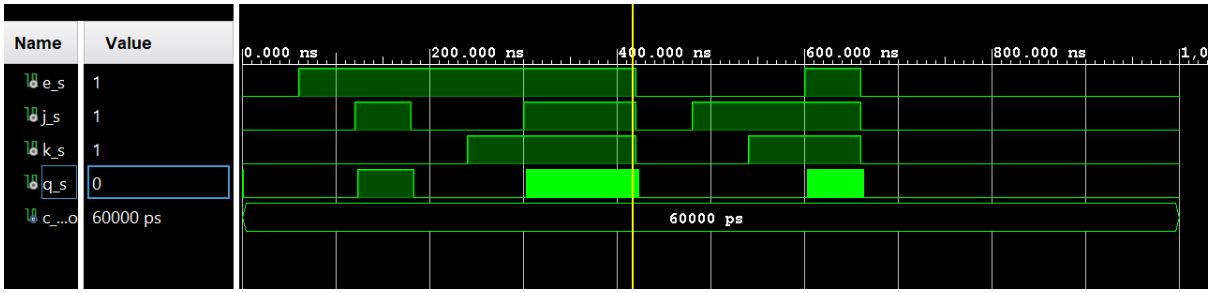
```
j <= '0';
wait 1ns;
```

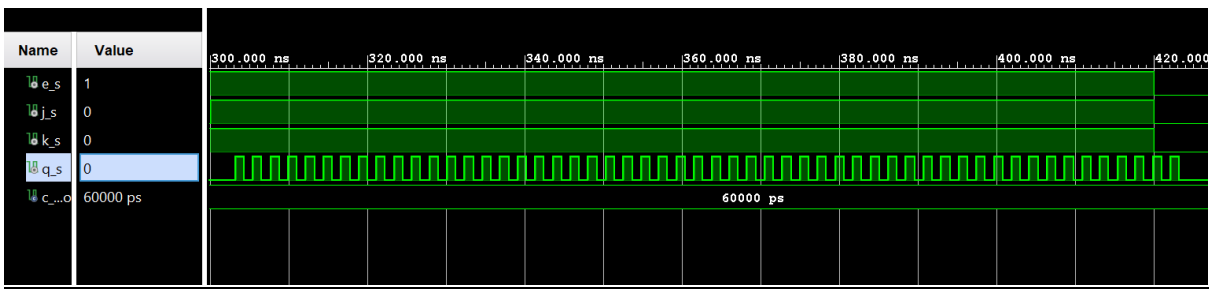
for system to detect a change or we can add a clock signal and process signals in every clock cycle. This will basically give us a flip-flop.



Post-Implementation

This simulation runs after the code is implemented over the selected chip. Rather than behavioral implementation, which ignores a lot of conditions, this test is real. It can be seen that the values of q are different than the behavioral test. It is resulted by delays and glitches in the real chip. It can also be seen that on the toggle mode the signal toggles between 2 values simultaneously. This is called "racing problem" can be seen in latches and single flip-flops.





Design Module

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

--entity and port declaration
entity jk_latch is
    Port (
    e : in std_logic;
    j : in std_logic;
    k : in std_logic;
    q : out std_logic);
end jk_latch;


architecture Behavioral of jk_latch is

signal q_int : std_logic := '0';

begin

--including q_int into the process is avoided since it causes a infinite combinational loop  on toggle state.

    process(e ,j, k)
    begin
        if (e = '1') then

            case  STD_LOGIC_VECTOR'(J & K) is

                when "00"=> --hold
                    null;

                when "01"=> --reset
                    q_int <= '0';

                when "10"=> -- set
                    q_int <= '1';

                when "11"=> --toggle to q_not
                    q_int <= not q_int;

                when others => -- for other logic types a bit can take such as 'U','X','W'
                    null;

             end case;

        end if;


    end process;

    q <= q_int; --q_int is transferred to q.



end Behavioral;
```

Test-Bench

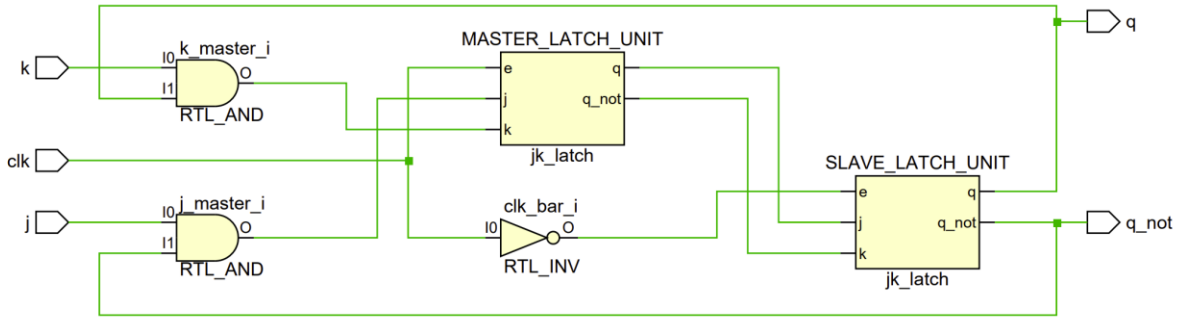
```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity jk_latch_tb is
--  Port ( );
end jk_latch_tb;

architecture Behavioral of jk_latch_tb is

--ports are defined
    component jk_latch
        Port (
        e : in std_logic;
        j : in std_logic;
        k : in std_logic;
        q : out std_logic);
    end component;

--signals are defined
    signal e_s : std_logic := '0';
    signal j_s : std_logic := '0';
    signal k_s : std_logic := '0';
    signal q_s : std_logic ;

--time variable
     constant c_period : time := 60 ns;


begin
    uut: jk_latch
--ports are assigned to signals
        port map(
            e => e_s,
```

```
            j => j_s,
            k => k_s,
            q => q_s
            );
    stim_proc: process

-- so now ne need to test the latch in 2 conditinons : enable is logic 1 and logic 0
--when enable - 1 it should act according to the behavioral when enable = 0 q should remain in the same value
    begin
    -- initialization
        e_s <= '0'; j_s <= '0'; k_s <= '0';
        wait for c_period;

        -- enable == 1
        e_s <= '1';
        wait for c_period;

        -- 1) SET || Q --> 1
        j_s <= '1'; k_s <= '0';
        wait for c_period;

        -- 2) HOLD || Q--> Q
        j_s <= '0'; k_s <= '0';
        wait for c_period;

        -- 3) RESET || Q --> 0
        j_s <= '0'; k_s <= '1';
        wait for c_period;

         -- 4) TOGGLE || Q --> Q'
        j_s <= '1'; k_s <= '1';
        wait for c_period;

         -- 5) TOGGLE || Q --> Q'
        j_s <= '1'; k_s <= '1';
        wait for c_period;

        -- enable == 0
        e_s <= '0'; j_s <= '0'; k_s <= '0';
        wait for c_period;


        -- 1) SET || Q --> 0 since enable is low
        j_s <= '1'; k_s <= '0';
        wait for c_period;


        -- 2) TOGGLE || Q --> 0 since enable is low
        j_s <= '1'; k_s <= '1';
        wait for c_period;


        -- 3) LET'S TURN ENABLE ON Q--> 1
        e_s <= '1';
        wait for c_period;

        e_s <= '0'; j_s <= '0'; k_s <= '0';
        wait for c_period;

        wait;
    end process stim_proc;

end architecture Behavioral;
```

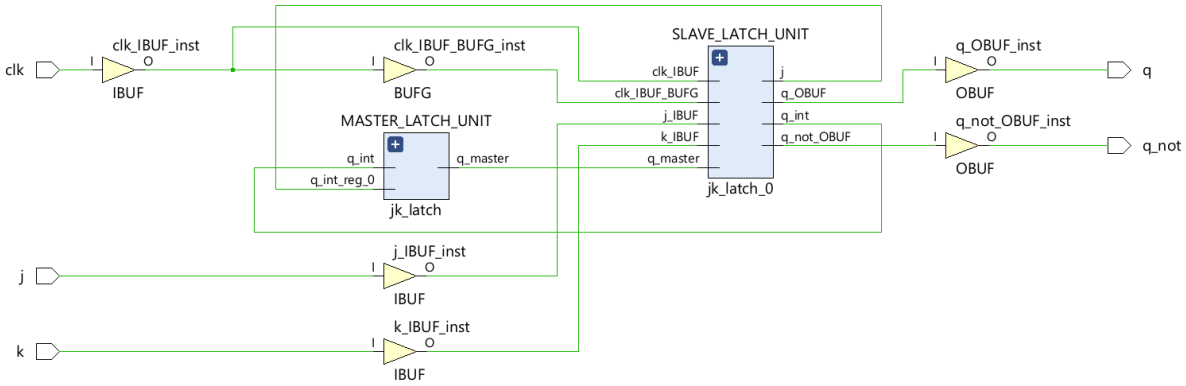
Part II)

In order to overcome the racing problem we have designed a master-slave JK Flip-Flop. On this model, the slave will only work when there

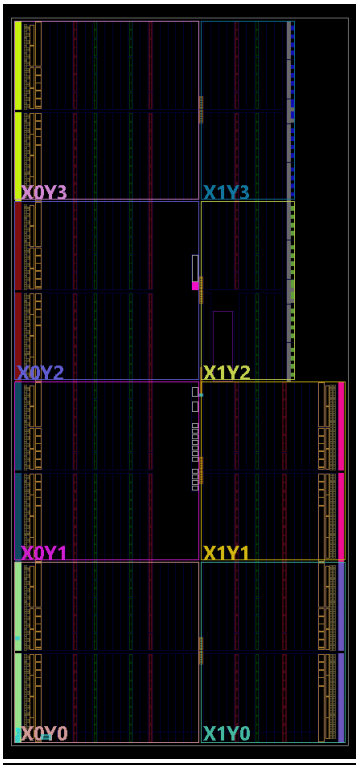
RTL Schematic



Implementation Schematic



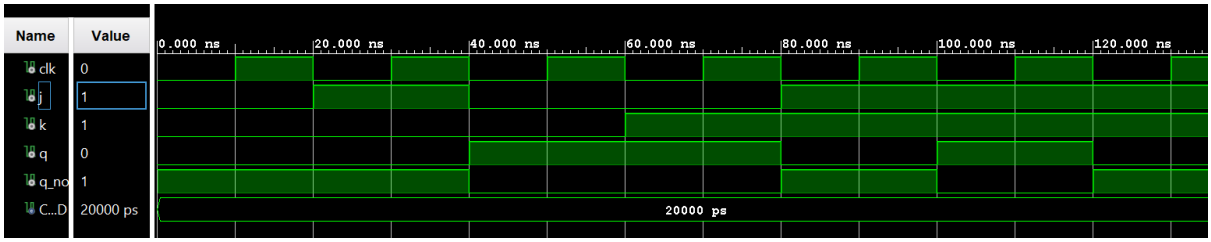
Implementation on FPGA /xc7k70tfbv676-1



Simulation

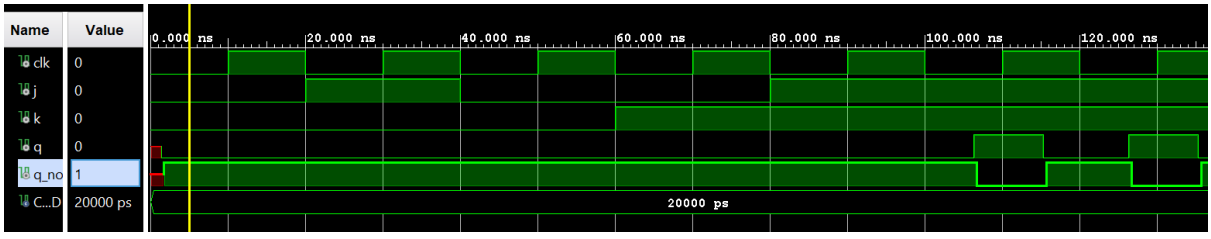
Behavioral

The behavioral simulation just works fine. We have overcome the consecutive toggle problem seen on the JK-Latch.



Post-Implementation

The conventional racing problem has been solved. However, the design used on JK-Latch is case insensitive it only looks up for the conditions, rather than rising or falling edges, this resulted shorter times for Q. Also, we can observe that as a result of delays. There is some interval JK could not affect.



We also have the Q as an unknown for the first couple of seconds, I think this is resulted because there is a phyical distnace between parts and this results in a delay in the declaration of Q..

Design Module

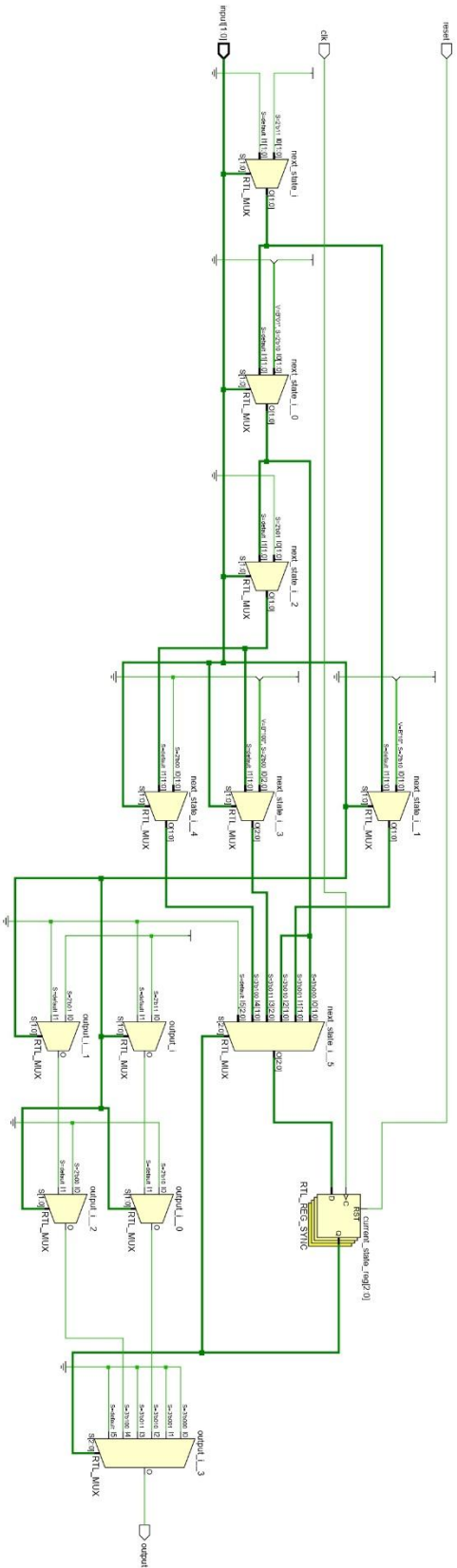
```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;



entity JK_FF is
    Port (
    clk : in std_logic;
    j : in std_logic;
    k : in std_logic;
    q : out std_logic;
    q_not : out std_logic);

end JK_FF;

architecture Structural of JK_FF is

    component jk_latch
        Port (
        e : in std_logic;
        j : in std_logic;
        k : in std_logic;
        q : out std_logic;
        q_not : out std_logic
        );
    end component;

--signals for master/slave

    signal clk_bar : std_logic;
    signal q_master : std_logic;
    signal q_master_not : std_logic;
    signal j_master : std_logic;
    signal k_master : std_logic;

    --a port declared as out can not be read wihin the same entity,
    --extra feedback signals are added to avoid this issue.

    signal q_fb : std_logic := '0';
    signal q_not_fb : std_logic := '1';

begin
    -- inverted clock for falling edge
    clk_bar <= not clk;

    --feedback logic inputs
    j_master <= j and q_not_fb;
    k_master <= k and q_fb;

    --master latch
    MASTER_LATCH_UNIT : component jk_latch
        Port map (
        e => clk,
        j => j_master,
        k => k_master,
        q => q_master,
        q_not => q_master_not
        );

    --slave latch
    SLAVE_LATCH_UNIT : component jk_latch
        Port map (
        e => clk_bar,
        j => q_master,
        k => q_master_not,
        q => q_fb,
        q_not => q_not_fb
        );
    q <= q_fb;
    q_not <= q_not_fb;

end Structural;
```

Test-Bench

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;


entity JK_FF_tb is
--  Port ( );
end JK_FF_tb;

architecture Behavioral of JK_FF_tb is

--component declaration
    component JK_FF is
        Port (
        clk : in std_logic;
        j : in std_logic;
        k : in std_logic;
        q : out std_logic;
        q_not : out std_logic);

    end component;

--signals connected to UUT
    signal clk : std_logic := '0';
    signal j : std_logic := '0';
    signal k : std_logic := '0';
    signal q : std_logic;
    signal q_not : std_logic;

-- clock period

    constant CLK_PERIOD : time := 20ns;

begin
    uut: JK_FF
        port map(
        clk => clk,
        j => j,
        k => k,
        q => q,
        q_not => q_not
        );

-- clock generation process
    CLK_GEN : process
    begin
        loop
            clk <= '0';
            wait for CLK_PERIOD/2; --clock is LOW
            clk <= '1';
            wait for CLK_PERIOD/2; -- clock is HIGH
        end loop;
   end process CLK_GEN;


--stimulus process and time diagram

    STIMULUS:process
    begin

    --initial state J = 0 - K = 0 - Q = 0
        j <= '0'; k <= '0';
        wait for CLK_PERIOD;

    --J = 1 - K = 0 ==> Q = 1
        j <= '1'; k <= '0';
        wait for CLK_PERIOD;

    --J = 0 - K = 0 ==> Q = 1
        j <= '0'; k <= '0';
        wait for CLK_PERIOD;

    --J = 0 - K = 1 ==> Q = 0
        j <= '0'; k <= '1';
        wait for CLK_PERIOD;

    --J = 1 - K = 1 ==> Q -> 1 -> 0 -> 1
        j <= '1'; k <= '1';
        wait for CLK_PERIOD;

        wait for CLK_PERIOD;

        wait for CLK_PERIOD;

        wait;

    end process STIMULUS;


end Behavioral;
```

Part III)

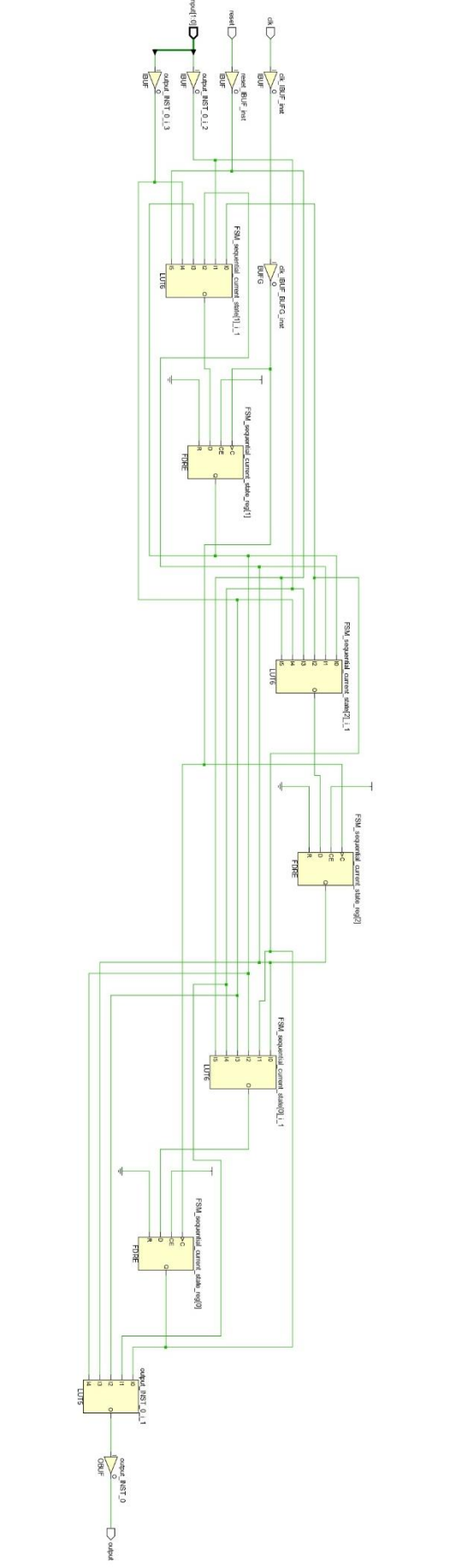
RTL Schematic



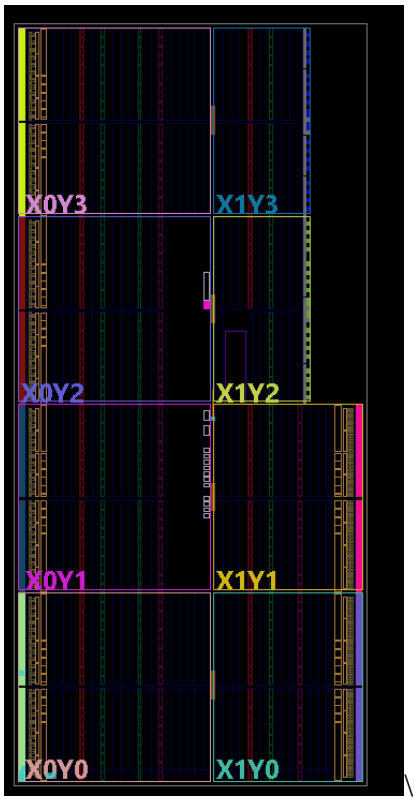
Implementation Schematic

It can be seen now we have 3 D FLip-Flops, which also satisfies:

$$\#\ \text{of Flip-Flops} = \log_2 \#\ of\ states$$



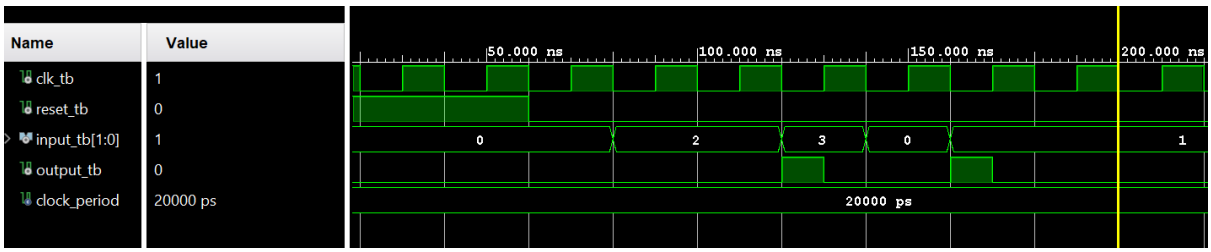
Implementation on FPGA /xc7k70tfbv676-1



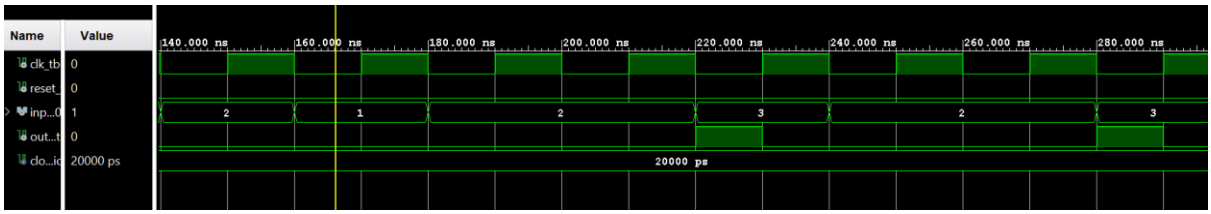
Simulation

Behavioral

Input: 10-10-11-00-01 (left to right)

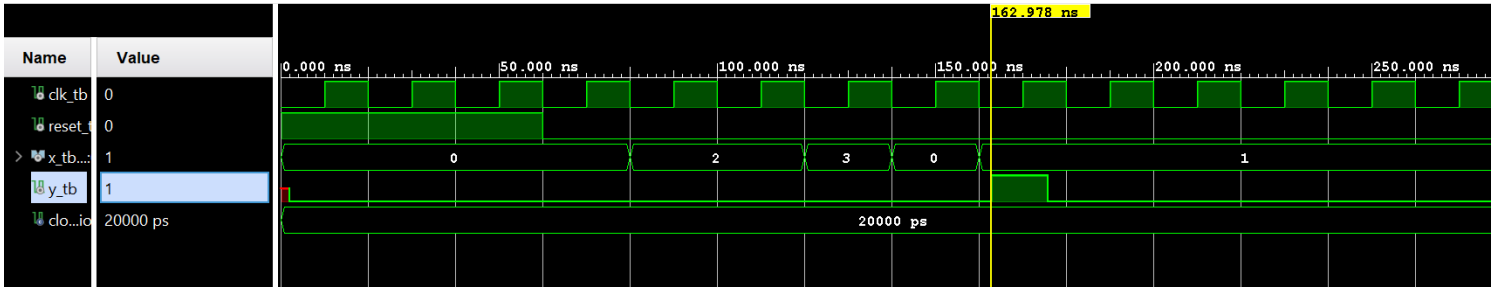


Input : 10-01-10-10-11-10-10-11(left to right)

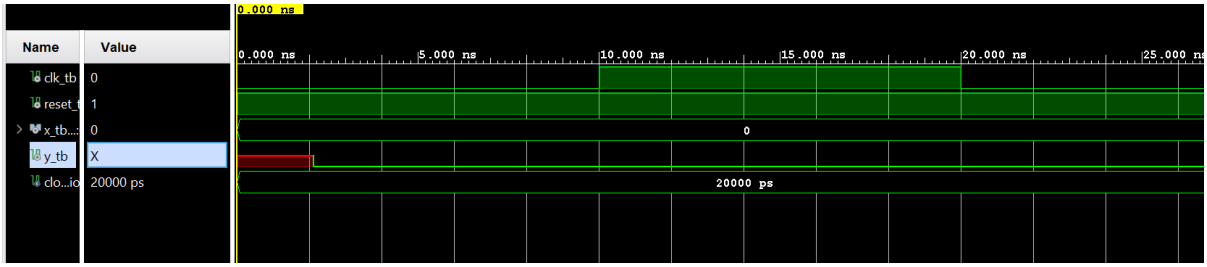


Post-Implementation

Input: 10-10-11-00-01 (left to right)



A delay of ~3ns can be seen, which resulted by the real time application of the design on the hardware. This time interval is caused by flip-flops, LUTs and wires.



Design Module

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;


entity pattern_detector_D_FF is

    Port (
    clk : in std_logic;
    reset : in std_logic;

    x : in std_logic_vector (1 downto 0) ;
    y : out std_logic
    );

end pattern_detector_D_FF;

architecture Behavioral of pattern_detector_D_FF is
    type states is (s0,s1,s2,s3,s4);

    signal current_state : states := s0;
    signal next_state : states;

begin

    process(clk) -- SENSITIVE TO CLK ONLY
        begin
            if rising_edge (clk) then
                -- Check for reset SYNCHRONOUSLY (on the clock edge)
                if (reset = '1') then
                    current_state <= S0;
                else
                    current_state <= next_state;
                end if;
            end if;
    end process;

    process(current_state, x )
        begin

        next_state <= s0;
        y <= '0';

            case current_state is
```

```
                when s0 =>
                    if (x = "10") then
                        y <= '0';
                        next_state <= s1;
                    elsif (x = "11") then
                        y <= '0';
                        next_state <= s3;
                    else
                        y <= '0';
                        next_state <= s0;

                    end if;

                when s1 =>
                    y <= '0';
                    if (x = "10") then
                        next_state <= s2;
                    elsif (x = "11") then
                        next_state <= s3;
                    else
                        next_state <= s0;

                    end if;

                when s2 =>
                    y <= '0';
                    if (x = "10") then
                        next_state <= s1;
                    elsif (x = "11") then
                        y <= '1';
                        next_state <= s3;
                    else
                        next_state <= s0;
                    end if;

                when s3 =>
                    y <= '0';
                    if (x = "00") then
                        next_state <= s4;
                    elsif (x = "01") then
                        next_state <= s0;
                    elsif (x = "10") then
                        next_state <= s1;
                    elsif (x = "11") then
                        next_state <= s3;
                    else
                        y <= '0';
                        next_state <= s0;
                    end if;

                when s4 =>
                    y <= '0';
                    if (x = "00") then
                        next_state <= s0;
                    elsif (x = "01") then
                        next_state <= s0;
                        y <= '1';
                    elsif (x = "10") then
                        next_state <= s1;
                    elsif (x = "11") then
                        next_state <= s3;
                    else
                        y <= '0';
                        next_state <= s0;
                    end if;

                 when others =>
                 y <= '0';
                 next_state <= s0;
                end case;


    end process;
end Behavioral;
```

Testbench

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;


entity pattern_detector_D_FF_tb is
end pattern_detector_D_FF_tb;

architecture Behavioral of pattern_detector_D_FF_tb is

        component pattern_detector_D_FF is
            Port (
                clk : in std_logic;
```

```
                reset : in std_logic;

                x : in std_logic_vector (1 downto 0) ;
                y : out std_logic
                );
         end component;

         --signals
         signal clk_tb : std_logic := '0';
         signal reset_tb : std_logic := '1';
         signal x_tb : std_logic_vector(1 downto 0) := "00";
         signal y_tb : std_logic;

         constant clock_period : time := 20 ns;

begin
    uut: pattern_detector_D_FF
        port map(
            clk => clk_tb,
            reset => reset_tb,
            x => x_tb,
            y => y_tb
        );

    -- clock generation process
    CLK_GEN : process
    begin
        loop
            clk_tb <= '0';
            wait for clock_period/2; --clock is LOW
            clk_tb <= '1';
            wait for clock_period/2; -- clock is HIGH
        end loop;
    end process CLK_GEN;

    --stimulus process and time diagram
    STIMULUS:process
    begin

        reset_tb <= '1';
        wait for 3 * clock_period;

        reset_tb <= '0';
        wait for clock_period;

        --report "Test Start" severity NOTE;

        --10-10-11-00-01
        x_tb <= "10"; wait for clock_period;
        x_tb <= "10"; wait for clock_period;
        x_tb <= "11"; wait for clock_period;
        x_tb <= "00"; wait for clock_period;
        x_tb <= "01"; wait for clock_period;

        wait;
    end process STIMULUS;
end Behavioral;
```

State Diagram